Image Processing Application Using Java and Spring Boot

By

P. Harshitha Devi

ASSDL Lab

Table of Contents

List of Figures

1. Introduction

This project focuses on building a complete **Image Processing Application** that enables pixel-level manipulation of digital images through a **Java and Spring Boot** REST API backend, with a **React + Vite** frontend for interactive use. The system implements a wide range of operations — brightness, contrast, blur, sharpness, rotation, flip, zoom, grayscale, background removal, shape detection, image layering, and undo — all processed entirely from scratch using Java's Buffered Image API, without any third-party image processing library.

The core design principle applied throughout every module is **Linear Indexing** — the mapping of 2D pixel coordinates (x, y) to a single 1D array index using the formula **index = y × width + x**. All pixel traversal, coordinate remapping, neighbourhood access for convolution, and offset-based composition rely on this technique.

2. Core Concept — Linear Indexing

Digital images are fundamentally two-dimensional structures where each pixel is addressed by a column **x** and a row **y**. However, in memory, pixel data is stored as a flat one-dimensional array. The mapping between these two representations is called **Linear Indexing**, and it is the foundational technique used throughout every image processing module in this project.

2.1 The Formula

Given an image of width W and height H, the linear index of a pixel at position (x, y) is:

$$\mathbf{index = y \times width + x}$$

y × width skips over all complete rows above the target row, and **x** moves to the correct column within that row. This formula assumes row-major order, which is the standard layout used by Java's BufferedImage.getRGB() and setRGB() methods.

2.2 Pixel Channel Extraction

Each pixel is packed as a 32-bit integer (ARGB). The individual colour channels are extracted using bit-shift operations and written back after modification:

$$\mathbf{R = (rgb >> 16) \ \& \ 0xFF}$$
$$\mathbf{G = (rgb >> 8) \ \& \ 0xFF}$$
$$\mathbf{B = rgb \ \& \ 0xFF}$$

The utility class **PixelUtil** centralises these operations, and **LinearIndexUtil** provides the toIndex(x, y, width), getX(index, width), and getY(index, width) helpers used across all service classes.

3. Image Processing Modules

Each module below describes the conceptual approach, the specific application of Linear Indexing, and the mathematical transformation involved. No third-party image processing libraries are used; all pixel manipulation is implemented from scratch using Java.

3.0 Image Upload

The Image Upload module accepts an image file from the frontend, converts it into a Buffered Image, and stores it in memory with a unique Image ID**.** The image dimensions (width and height) are extracted and returned to the frontend. The generated Image ID is used for all subsequent image processing operations.

An image is represented as a matrix of pixels:

$$I = \{P(x,y)\}$$

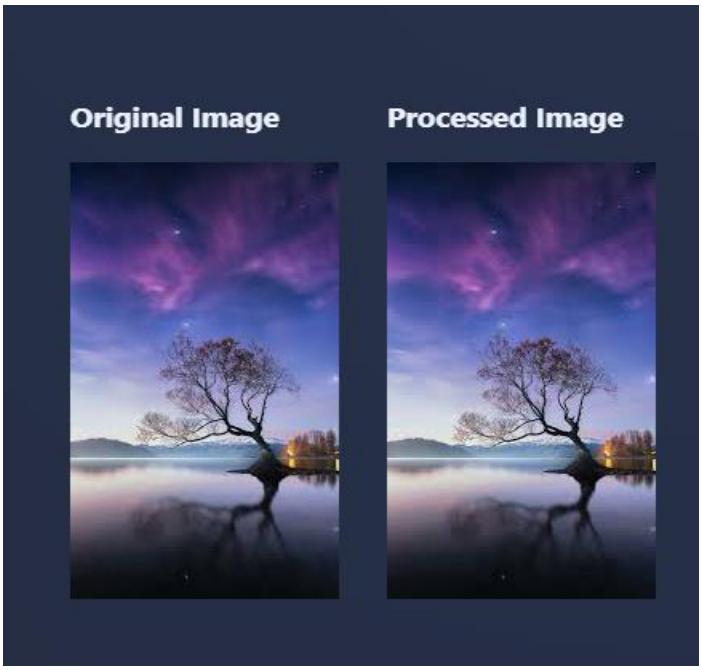
Each pixel contains RGB values:

$$P(x,y) = (R,G,B)$$

The uploaded image is stored as:

$$ImageStore[ImageID] = I$$

Figure 1: Image Upload



3.1 Brightness Adjustment

Concept & Application

Brightness adjustment adds a constant delta value (−100 to +100) to the RGB channels of each pixel. The updated values are clamped to the range **[0, 255]** before reconstructing the output image.

for i in 0 .. (width × height − 1):

R' = clamp(R + delta), G' = clamp(G + delta), B' = clamp(B + delta)

clamp(v) = max(0, min(255, v))

Linear Indexing is used in its simplest form: sequential traversal of every pixel with no coordinate transformation, making it an O(N) operation over all N = width × height pixels.

Figure 2: Brightness Adjustment

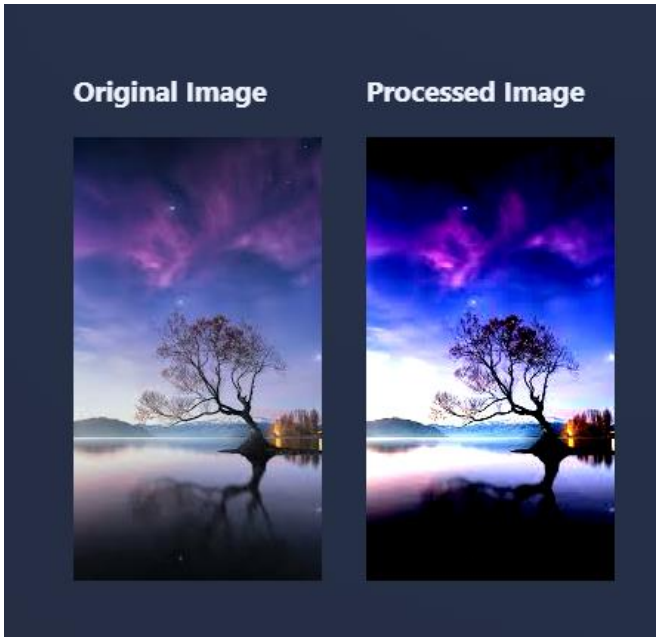
3.2 Contrast Adjustment

Concept & Application

Contrast adjusts pixel values around the midpoint (128) using the formula: channel′ = clamp(factor × (channel − 128) + 128)**.**

The pixel array is traversed sequentially, with each pixel's RGB values read, adjusted using the contrast formula, and written back to the same position.

Figure 3: Contrast Adjustment



3.3 Grayscale Conversion

Concept & Application

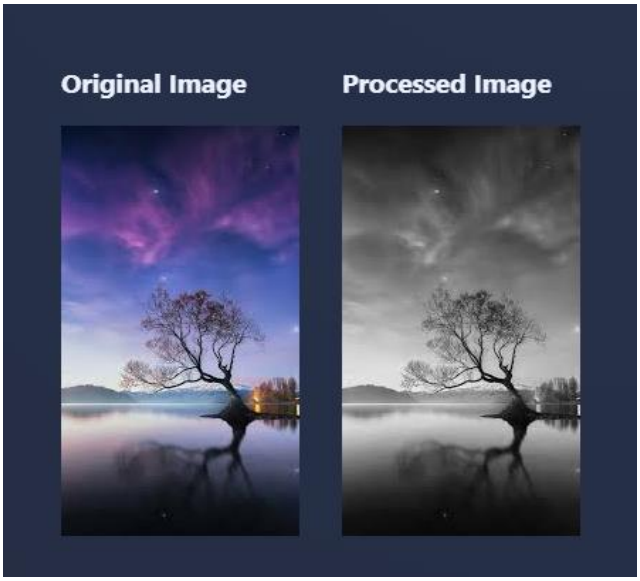
Grayscale conversion converts an image to shades of gray by averaging the three RGB channels.The resulting value is assigned identically to R, G, and B, producing a neutral grey tone:

$$gray = (R + G + B) / 3$$

$$R' = G' = B' = gray$$

The pixel array is traversed once, with each pixel updated at the same index position.

Figure 4: Grayscale Conversion



3.4 Horizontal Flip

Concept & Application

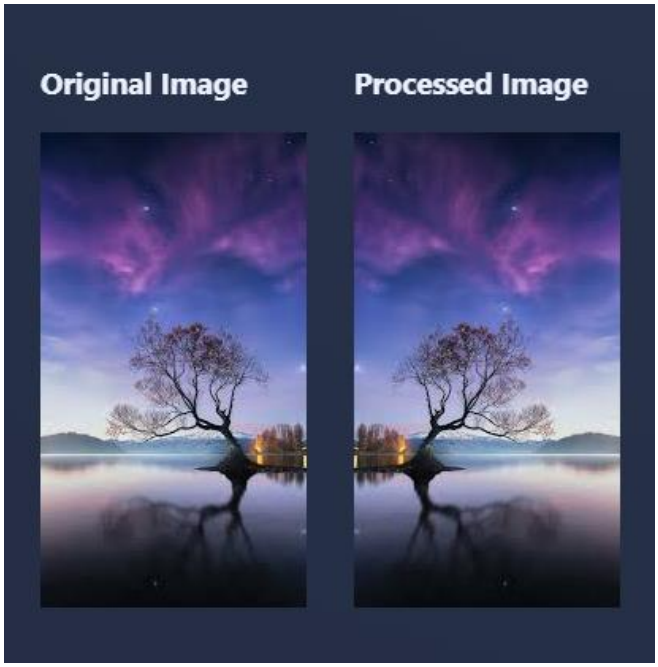
A horizontal flip mirrors every pixel across the vertical centre line. Pixel (x, y) moves to (width − 1 − x, y). The row remains the same; only the column address is mirrored:

$$\text{sourceIndex} = y \times \text{width} + x$$

$$\text{targetIndex} = y \times \text{width} + (\text{width} - 1 - x)$$

Linear indexing maps 2D coordinates directly to array positions.

Figure 5: Horizontal Flip



3.5 Vertical Flip

Concept & Application

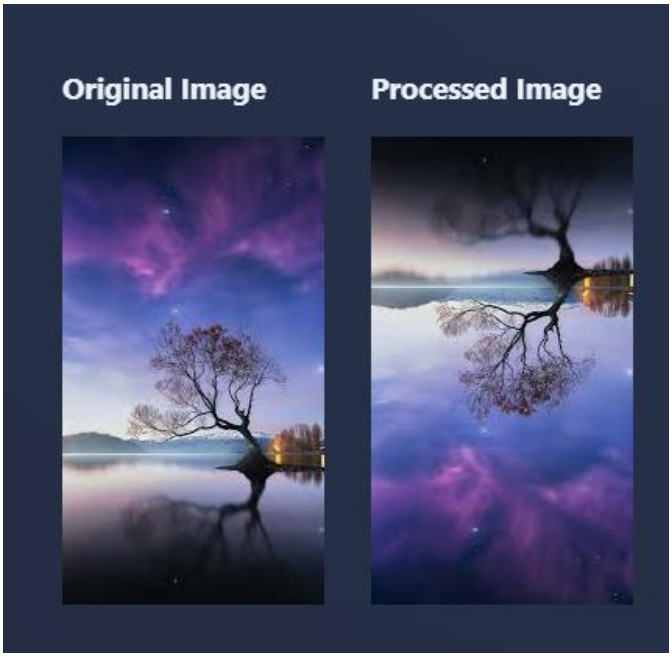
A vertical flip mirrors pixel (x, y) to position (x, height − 1 − y). The column x stays fixed; the row is mirrored:

$$\text{sourceIndex} = y \times \text{width} + x$$

$$\text{targetIndex} = (\text{height} - 1 - y) \times \text{width} + x$$

Subtracting the row from (height − 1) and multiplying by width gives the correct offset into the 1D array for the flipped row, with no additional bookkeeping.

Figure 6: Vertical Flip



3.6 Image Rotation

Concept & Application

The project supports 90°, 180°, and 270° rotations. Each remaps pixel coordinates and converts them to a linear index in the result array. Output dimensions swap (width × height becomes height × width) for 90° and 270°. The source pixel is always read as sourceIndex = y × width + x; the new coordinates are computed and immediately written to the target 1D array without constructing a 2D output structure.

Rotation 90° (clockwise):

$$newX = height - 1 - y, \quad newY = x$$
$$targetIndex = newY \times height + newX$$

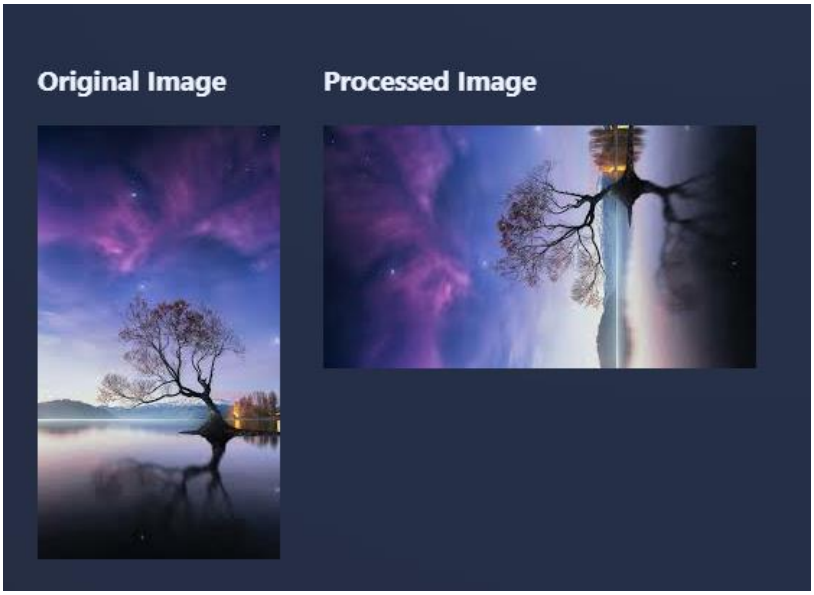
Rotation 180°:

$$newX = width - 1 - x, \quad newY = height - 1 - y$$
$$targetIndex = newY \times width + newX$$

Rotation 270° (clockwise):

$$newX = y, \quad newY = width - 1 - x$$
$$targetIndex = newY \times height + newX$$

Figure 7: Image Rotation



3.7 Image Zooming

Concept & Application

Zooming enlarges the image by mapping each output pixel to the nearest source pixel using reverse mapping (nearest-neighbour interpolation).

$$newWidth = width \times factor, \quad newHeight = height \times factor$$

$$sourceX = (int)(x / factor), \quad sourceY = (int)(y / factor)$$

$$sourceIndex = sourceY \times originalWidth + sourceX$$

$$targetIndex = y \times newWidth + x$$

A factor greater than 1.0 zooms in, while a factor less than 1.0 zooms out. Linear indexing maps pixels between the source and output arrays.

Figure 8: Image Zooming (Zoom In)

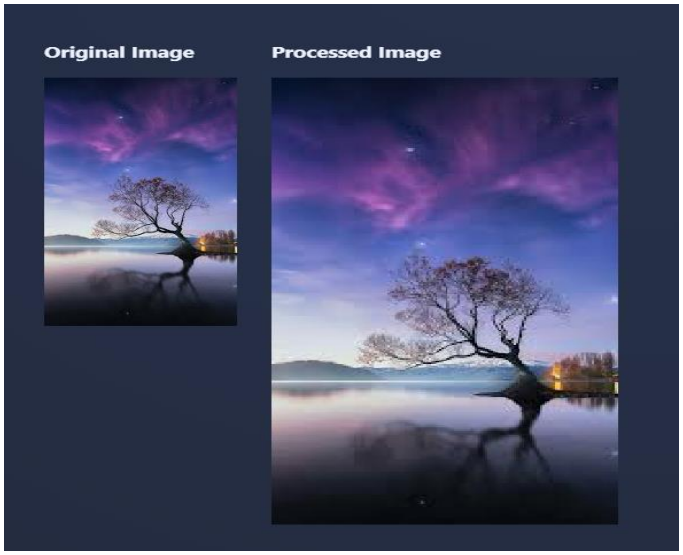
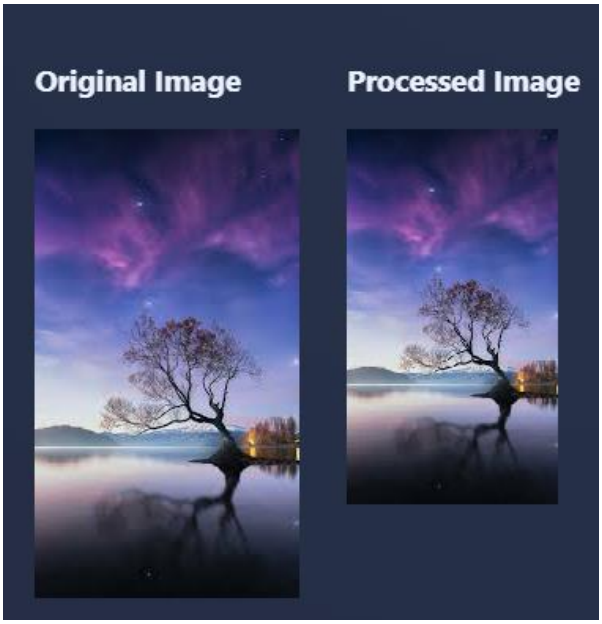


Figure 9: Image Zooming (Zoom Out)



3.8 Blur Operation

Concept & Application

Blur uses a box filter that replaces each pixel with the average of its surrounding kernelSize × kernelSize neighbourhood. Each neighbour is accessed using Linear Indexing:

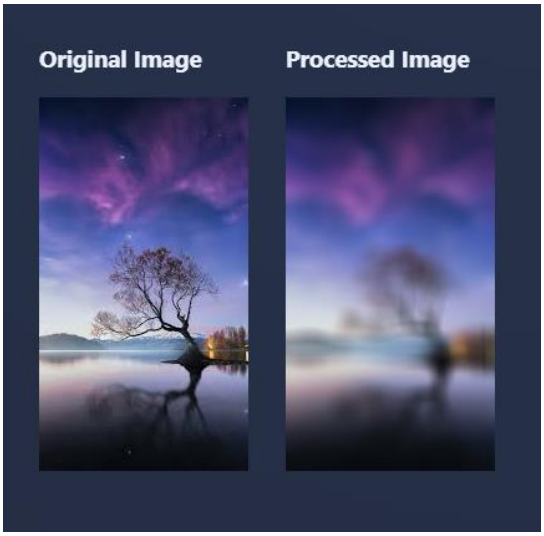
$$\text{neighbourIndex} = (y + ky) \times \text{width} + (x + kx)$$

$$R' = \text{sum}(R\_\text{neighbours}) / \text{count} \quad \text{(same for G, B)}$$

$$\text{targetIndex} = y \times \text{width} + x$$

Pixels near the image boundary only average available neighbours (partial kernel), and the count adjusts accordingly to avoid out-of-bounds access.

Figure 10: Blur Operation



3.9 Sharpness Enhancement

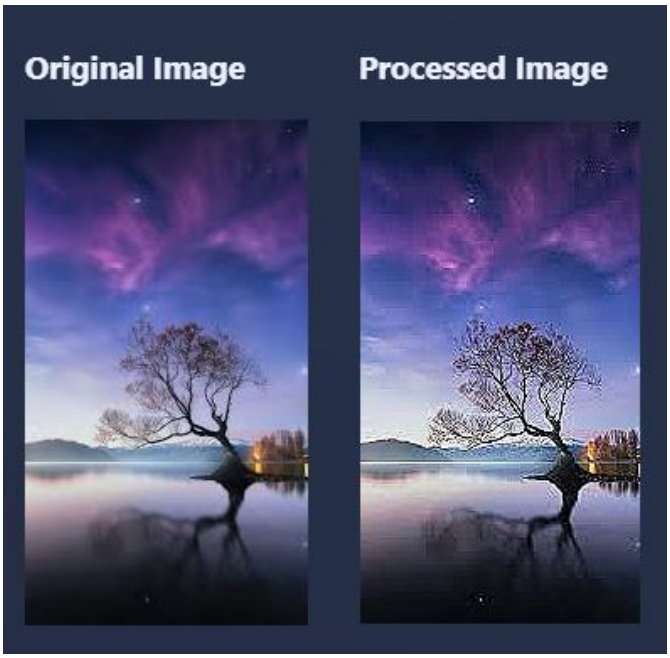
Concept & Application

Sharpening uses an unsharp-mask approach based on a 4-neighbour Laplacian approximation. The centre pixel is reinforced by its difference from the average of its four cardinal neighbours:

$$\text{new} = \text{clamp}(\ \text{center} + \text{factor} \times (\text{center} - \text{average}(\text{top}, \text{bottom}, \text{left}, \text{right}))\ )$$

Each neighbour at (x + kx, y + ky) is fetched via Linear Indexing (neighbourIndex = ny × width + nx). The result is clamped to [0, 255] and written to targetIndex = y × width + x. Border pixels (1 px edge) are left unprocessed.

Figure 11: Sharpness Enhancement



3.10 Image Layering

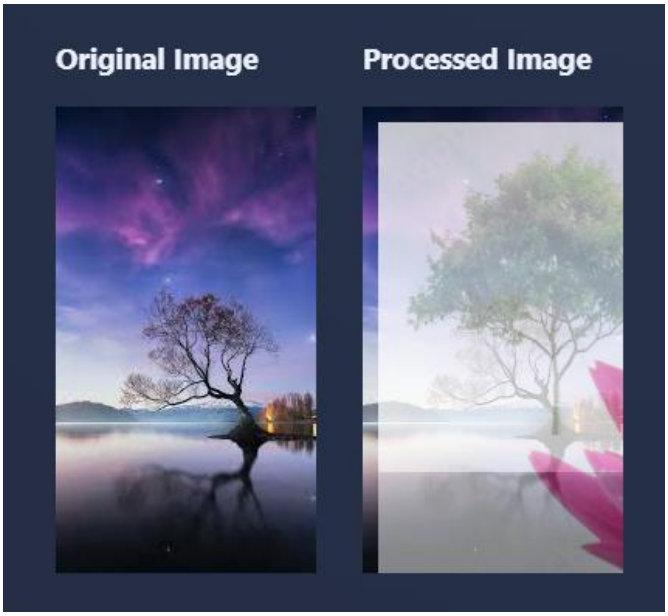
Concept & Application

Layering places a foreground image on top of a background image starting at pixel offset (startX, startY). For each pixel (x, y) of the foreground, the corresponding background position is (startX + x, startY + y). Both positions are converted to linear indices:

$$\text{overlayIndex} = y \times \text{overlayWidth} + x$$
$$\text{backgroundIndex} = (\text{startY} + y) \times \text{backgroundWidth} + (\text{startX} + x)$$

The foreground is composited at 50% transparency using AlphaComposite.SRC_OVER with alpha 0.5f via Java's Graphics2D. A bounds check is applied before each write to ensure the target pixel lies within the background dimensions.

Figure 12: Image Layering



3.11 Background Removal

Concept & Application

The algorithm samples the background colour from the top-left pixel (0, 0). For every other pixel, it computes the Manhattan distance in RGB colour space between that pixel and the sampled background colour:

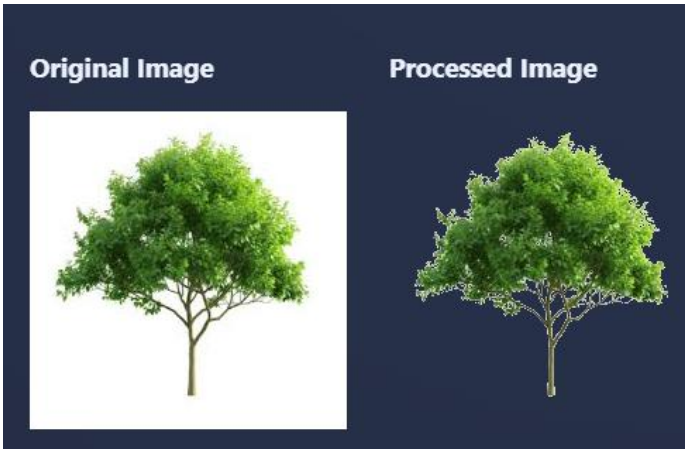
$$d = |R - bgR| + |G - bgG| + |B - bgB|$$

$$\text{if } (d \leq \text{threshold}) \rightarrow \text{transparent (ARGB = 0x00000000)}$$

$$\text{else} \rightarrow \text{keep original pixel}$$

The output uses TYPE_INT_ARGB to support transparency. Each pixel's source position is computed as **y × width + x** — a direct application of the Linear Index formula.

Figure 83: Background Removal



3.12 Shape Detection

Concept & Application

Shape detection is a multi-stage pipeline that identifies the dominant shape in the image:

Thresholding: Each pixel brightness is computed as (R + G + B) / 3. Pixels with brightness < 128 are treated as dark (foreground); others as background. The traversal uses the standard Linear Index formula.

Bounding Box: The min/max x and y coordinates of all dark pixels are tracked to compute the bounding box width and height.

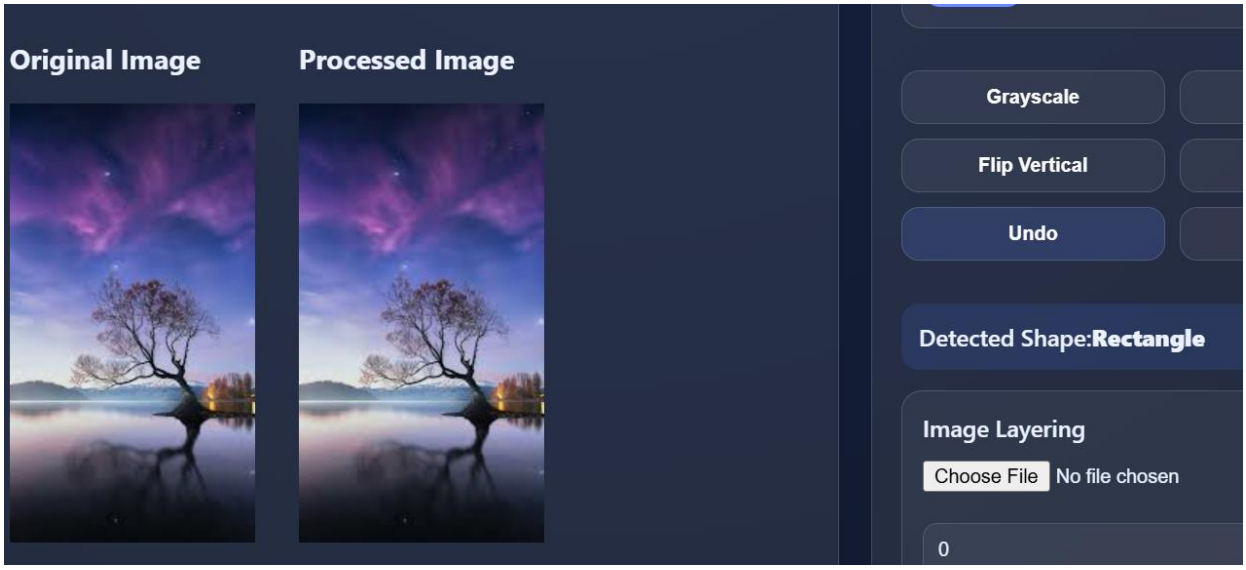
Shape Classification: The aspect ratio of the bounding box determines the class:

$$\text{ratio} = \text{shapeWidth} / \text{shapeHeight}$$

$$\text{if } (0.9 < \text{ratio} < 1.1) \rightarrow \text{Square}$$

$$\text{else} \rightarrow \text{Rectangle}$$

Figure 94: Shape Detection



4. System Architecture

The application is structured as a two-tier system: a **Spring Boot REST API** backend and a **React + Vite** frontend. All image processing logic resides exclusively in the backend.

4.1 Backend Structure

ImageStore: A Spring `@Component` holding two `ConcurrentHashMap` maps — one from UUID → `BufferedImage`, and one recording parent–child relationships for undo support. All images are in JVM heap memory; there is no file system or database persistence.

Service Layer: A single ImageServiceImpl class (implementing the ImageService interface) contains one method per operation. Each reads from ImageStore, applies the transformation, saves the result as a new entry, and returns a DTO containing the new imageId, dimensions, and a status message.

ImageController: A single @RestController mapped to /api exposes all endpoints. CORS is enabled for http://localhost:5173. The controller never performs image processing; it only delegates to the service layer.

Utility Classes: ImageUtil (file I/O), PixelUtil (channel extraction & packing), and LinearIndexUtil (1D↔2D coordinate mapping) are shared across all service methods.

4.2 Frontend Structure

App.jsx: Root component. Owns the imageId state and coordinates all child components.

UploadImage.jsx: File picker that sends the chosen image via POST /api/upload and propagates the returned imageId to App.

ImageViewer.jsx: Renders <img src="http://localhost:8080/api/image/{imageId}">. Automatically refreshes whenever imageId changes.

ImageControls.jsx: All processing controls — sliders for brightness, contrast, blur, sharpness, zoom, background threshold; a dropdown for rotation angle; action buttons for grayscale, flip, shape detection, undo, and download; and a layering section with file input and offset fields.

imageService.js: Service layer that encapsulates all Axios calls. Components never call Axios directly.

4.3 API Endpoints

Method	Endpoint	Parameter(s)	Operation
POST	/upload	file (multipart)	Upload image
GET	/image/{id}	—	Retrieve image for preview
GET	/download/{id}	—	Download image (attachment)
POST	/{id}/brightness	brightness (int)	Brightness adjustment
POST	/{id}/contrast	contrast (double)	Contrast adjustment
POST	/{id}/blur	kernelsize (int)	Box-blur filter
POST	/{id}/sharpness	factor (double)	Sharpness enhancement
POST	/{id}/rotate	angle (90 / 180 / 270)	Rotation

POST	/{id}/flip	direction (horizontal / vertical)	Flip
POST	/{id}/zoom	factor (double)	Zoom in / out
POST	/{id}/grayscale	—	Grayscale conversion
POST	/{id}/background-remove	threshold (int)	Background removal
POST	/{id}/shape-detection	—	Shape detection
POST	/layer	backgroundImageId, foregroundFile, xOffset, yOffset	Image layering
POST	/{id}/undo	—	Undo last operation

5. Expected Workflow

The application follows a stateless request–response workflow where each processing operation produces a new immutable image version:

#	Step	Description
1	**Upload**	User selects an image file. The frontend sends it via POST /api/upload. The backend generates a UUID, stores the BufferedImage in memory, and returns the imageId.
2	**Preview**	The imageId is stored in React state. ImageViewer renders the image by fetching GET /api/image/{imageId}.
3	**Apply Operation**	User adjusts a slider or selects an option and clicks Apply. The frontend calls the corresponding service function (e.g. adjustBrightness(imageId, value)).
4	**Backend Processing**	The controller receives the request, delegates to ImageServiceImpl. The service fetches the source image from ImageStore, runs the pixel transformation, saves the result as a new entry (recording the parent), and returns a DTO with the new imageId.
5	**Preview Update**	The frontend updates imageId state to the new value. ImageViewer re-renders with the updated image URL — showing the processed result.
6	**Undo**	If the user clicks Undo, POST /{id}/undo is called. The service looks up the parent imageId and returns it. The frontend reverts imageId to the parent, restoring the previous image.
7	**Download**	The final image is fetched from GET /api/download/{imageId} as a binary blob and saved to disk via a programmatic anchor click.

6. Conclusion

This project implements a complete image processing system in Java using pixel-level manipulation without external image processing libraries. All operations are based on Linear Indexing, which maps 2D pixel coordinates to a 1D array for efficient pixel access. The modular design of ImageServiceImpl, along with shared utility classes, keeps the implementation reusable and maintainable. Combined with the Spring Boot backend and React frontend, the application provides real-time image processing with live preview and download support.